IF2210

Pemrograman Berorientasi Objek

UTS SEMESTER II – 2009/2010

IF2032

15 Maret 2010

1. Tuliskan implementasi operator << untuk kelas Stack yang memiliki spesifikasi di bawah ini. Operator << berfungsi untuk menerima data dari input stream dan menambahkannya ke stack. Tentukan apakah operator tersebut diimplementasikan sebagai member function atau bukan.

```
#ifndef _STACK_H
#define _STACK_H

class Stack {

public:
//deklarasi ctor, cctor, dtor & assignment operator

   void push(int x);
   int pop();

private:
   int *data; //array untuk menampung elemen pada Stack
   int currentCapacity; //ukuran alokasi array pada data
   int topStack; //posisi stack saat ini
};
#endif //_STACK_H
```

2. Jelaskan masalah yang mungkin muncul pada penggunaan multiple inheritance
3. Tuliskan hasil eksekusi kode berikut:

```
int main()
{
   int m = 44;
   cout << "m = " << m << ", m++ = " << m++ << ", ++m = " << ++m << endl;
}
```

4. Jelaskan apa yang salah pada kode berikut:

```
#include <iostream>

int f(int &c) {
   return c++;
}

int main() {
   std::cout << "f(5) = " << f(5) << std::endl;
}
```

5. Tuliskan hasil eksekusi kode berikut ini (problem ctor, cctor, passing by referance), dan jelaskan jawabannya.

```
#include "iostream"

class X {
  public:
    X():value(0) { std::cout << "X::ctor" << std::endl; }
    X(const X& x):value(x.value) {
      std::cout << "X::cctor" << std::endl;
```

```
    }
    ~X() {
      std::cout << “X::dtor” << std::endl;
    }
    X& operator=(const X& s) {
      this->value = s.value;
      return *this;
    }
    void setValue(int x) {
      this->value = x;
    }
  private:
    int value;
};

X f1(X x) {
  std::cout << “f1()” << std::endl;
  return x;
}

X f2(X &x) {
  std::cout << “f2()” << std::endl;
  return x;
}

X& f3(X &x) {
  std::cout << “f3()” << std::endl;
  return x;
}

X f4(X *x) {
  std::cout << “f4()” << std::endl;
  return *x;
}

int main() {
 X x;
 X y;
 y = f1(x);
 y = f2(x);
 y = f3(x);
 y = f4(&x);
}
```

6. Jelaskan apa yang salah dari kode berikut, dan tuliskan hasil eksekusi programnya.

```
#include <iostream>
#include <string.h>

class array {
       protected:
              int size;
              int *data;
       public:
              array(const int i_size): size(i_size), data(new int[size]) {
                     std::cout << "cctor" << std::endl;
                     memset(data,'\0',size * sizeof(data[0]));
              }
              array(const array &src) {
                     std::cout << "cctor" << std::endl;
                     size = src.size;
```

```
                        data = new int[src.size]
                        memcpy(data, src.data, size * sizeof(data[0]));
                }
                virtual ~array(void) {
                        std::cout << "dtor" << std::endl;
                        delete []data;
                        data = NULL;
                }
                array &operator= (const array &src) {
                        std::cout << "operartor=" << std::endl;
                        delete []data;
                        size = src.size;
                        data = new int[src.size];
                        memcpy(data, src.data, size * sizeof(data[0]));
                        return (*this);
                }
                int &operator [](const unsigned int item) {
                        std::cout << "operator[]" << std::endl;
                        return data[item];
                }
};

void copy_array(array &dest, const array &src) {
        std::cout << "copy array" << std::endl;
        dest = src;
}

int main() {
        array a(30), b(2);
        a[0] = 0;
        a[1] = 1;
        a[2] = 2;
        copy_array(b, a);
        copy_array(a, a);
        std::cout << "a[0]: " << a[0] << ", a[1]: " << a[1] << ", a[2]: " <<
a[2]
                << std::endl;
        std::cout << "b[0]: " << b[0] << ", b[1]: " << b[1] << ", b[2]: " <<
b[2]
                << std::endl;
        return(0);
}
```

7. Jelaskan apa yang salah pada kode berikut ini

```
Class X
{ protected:
int a;
};
class Y: public X
{ public:
void set(X x, int c) {x.a = c;}
};
```

8. Jelaskan apa yang dimaksud dengan virtual function, dan berikan contoh penggunaannya.

UTS SEMESTER II – 2012/2013

IF2032/Pemrograman Berorietasi Objek

Tanggal: 20 Maret 2013, 120 menit, closed book

Soal 1. Konsep (Bobot: 15%)

a. Jelaskan apa yang dimaksud dengan **kelas abstrak** dan berikan ilustrasi dalam bentuk contoh kode dalam bahasa C++.
b. Jelaskan apa yang dimaksud dengan **polymorphism** dan berikan ilustrasi dalam bentuk contoh kode dalam bahasa C++.
c. Jelaskan apa yang dimaksud dengan **exception** dan **assertion** dan berikan ilustrasi dalam bentuk contoh kode dalam bahasa C++.

Soal 2. Eksekusi Program – C++ (Bobot: 15%)

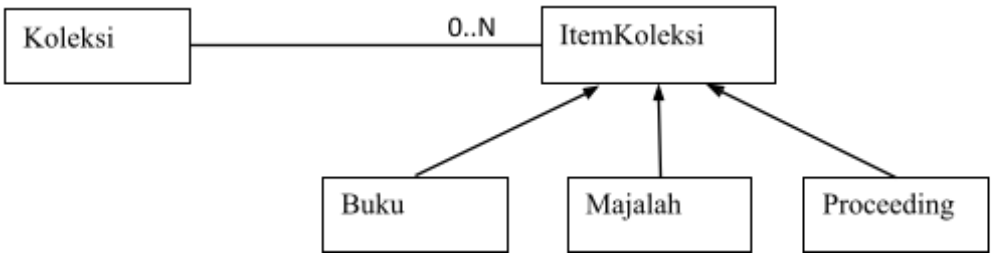
Tuliskan hasil eksekusi kode berikut dengan cara mengisikan titik-titik pada bagian komentar program di bawah ini. **Jika tak ada output/hasil eksekusi, tuliskan "Tidak ada ****"**

```
#include <iostream>
Using namespace std;
class A {
protected:
      int x;
public:
      A()                       { x = 0; cout << "1a "; }
      A(int _x)                 { x = _x; cout << "1b "; }
      A(const A& _A)            { x = A.x; cout << "1c "; }
      ~A()                      { cout << "1d "; }
      void f1()                 { cout << "A::f1()" << endl; }
      virtual void f2()         { cout << "A::f2()" << endl; }
};

class B: protected A {
protected:
      double y;
public:
      B()                       { y = 0; cout << "2a "; }
      B(int a, double b): A(a)  { y = b; cout << "2b "; }
      B(const B& _B)            { y = B.y; cout << "2c "; }
      ~B()                      { cout << "2d "; }
      virtual void f1()         { cout << "B::f1()" << endl; }
      virtual void f2()         { cout << "B::f2()" << endl; }
int main() {
      // Di awal program (1.0) ..........
      B b1(1, 2.0);       // (1.1) ..........
      B b2;               // (1.2) ..........
      B b3 = b2;          // (1.3) ..........
      A a1(10), a2;       // (1.4) ..........
      a2 = a1;            // (1.5) ..........

      A *pa1;
      B *pb1;
      pa1 = &a1;
      pa1 -> f1();        // (1.6) ..........
      pa1 -> f2();        // (1.7) ..........
      pa1 = &b1;
      pa1 -> f1();        // (1.8) ..........
```

```
        pa1 -> f2();        // (1.9) ..........
        pb1 = &b1;
        pb1 -> f1();        // (1.10) ..........
        pb2 -> f2();        // (1.11) ..........

        // sesaat sebelum keluar program (1.12) ..........
}
```

Soal 3. Implementator kelas – C++ (Bobot: 40%)
Perhatikan bahwa, secara prinsip persoalan berikut mirip dengan salah satu Soal UTS IF2032 yang lalu.

Sebuah sistem **perpustakaan** berbayar memiliki **koleksi** berbagai **item koleksi** perpustakaan, yang terdiri atas **buku, majalah,** dan **proceeding** (kumpulan paper konferensi/seminar). Setiap item koleksi memiliki identitas/id yang unik dan untuk satu judul diasumsikan hanya ada 1 eksemplar item. Untuk memodelkan koleksi yang dimiliki perpustakaan, telah didesain kelas diagram di bawah ini. Perhatikan bahwa dalam persoalan ini, anggota yang dimiliki perpustakaan tidak dimodelkan dan tidak menjadi bagian dari sistem ini.



Perhatikan diagram di atas. Koleksi terdiri atas nol atau lebih ItemKoleksi. ItemKoleksi adalah sebuah kelas abstrak, dan diturunkan menjadi kelas konkret Buku, Majalah, dan Proceeding. Jumlah koleksi yang dapat dikelola, maksimum 1000 item koleksi. ItemKoleksi memiliki atribut id (integer), judul (string), dan tahun terbit (integer), sedangkan Majalah memiliki atribut tambahan berupa bulanTerbit (integer).

ItemKoleksi memiliki fungsi dan prosedur sbb.:

1. **Fungsi abstrak HitungBiayaPeminjaman (<u>input</u> lamaPinjam: <u>integer</u>)** yang menghitung dan mengembalikan biaya peminjaman dengan kebijakan yang berbeda tergantung jenis itemnya sbb:

No	Jenis ItemKoleksi	Biaya Peminjaman
1	Buku	LamaPeminjaman*Rp. 500
2	Majalah	LamaPeminjaman*RP. 1500*(1+(1/TahunTerbit))
3	Proceeding	Rp. 5000 (tidak tergantung dengan lama peminjaman)

2. **Prosedur PrintInfo()**, yang mencetak nilai semua atribut yang dimiliki oleh item koleksi.

Koleksi memiliki prosedur sbb.:

1. **procedure AddItem (<u>input</u> item: ItemKoleksi)**, yang menambahkan item baru pada table item yang dikelola oleh koleksi dan merupakan **array static**.
2. **Procedure Print()**, yang mencetak nilai seluruh atribut yang dimiliki oleh semua item yang dikelola oleh koleksi tersebut.

Tugas Anda adalah:

1. Tuliskan **Koleksi.h, ItemKoleksi.h, dan Majalah.h,** yang merepresentasikan desain kelas di atas. Anda harus menambahkan deklarasi **empat sekawan** dari ketiga kelas tersebut. Tambahkan atribut lain dan method lain dari ketiga kelas tersebut, apabila diperlukan.
2. Anggaplah semua kelas yang didesain sudah diimplementasikan secara lengkap, buatlah kelas **mainLib.cpp** yang memiliki perilaku sebagai berikut:
   a. Menambahkan Item koleksi berupa Buku dengan ItemId = 1, Judul = "Object Oriented Programming", tahun terbit = 2010.
   b. Menambahkan Item koleksi berupa Majalah dengan ItemId = 2, Judul = "Informatics Meagazine", tahun = 2012, bulan terbit = 2.
   c. Mencetak semua item (buku, majalah, proceeding yang dikelola koleksi).
3. Implementasikan **Majalah.cpp** secara lengkap. Perhatikan bahwa Anda juga harus mengimplementasi fungsi **HitungBiayaPeminjaman**, dan meng-*override* **procedure PrintInfo**.

Soal 4. Desain Kelas – C++ (Bobot: 20%)

Sebuah *drawing application* digunakan untuk menggambar beberapa bangun 2 dimensi, yaitu **segitiga**, **segiempat**, dan **lingkaran**. Ketiga jenis bangun mempunyai atribut yang berbeda-beda: segitiga mempunyai tiga sisi, segiempat punya empat sisi, dan lingkaran punya sebuah jari-jari (semua sisi dan jari-jari bertipe *double*). Tapi untuk semua jenis bangun, bisa dihitung keliling dan luasnya (walaupun tentu saja dengan rumus yang berbeda-beda) serta dapat ditampilkan gambarnya seperti apa di layar.Ada beberapa jenis segitiga yang spesifik, yaitu segitiga sama kaki, sama sisi, dan siku-siku. Perbedaan ketiganya hanyalah pada kriteria dari sisi-sisinya. Untuk segitiga sama kaki, maka panjang dua buah sisi harus sama. Untuk segitiga sama sisi, panjang semua sisi harus sama. Untuk segitiga siku-siku, jika a, b, dan c adalah sisi-sisinya, harus memenuhi $a^2 + b^2 = c^2$. Tidak ada operasi dan atribut spesifik yang berlaku untuk masing-masing jenis segitiga. Untuk bangun segiempat, ada operasi khusus yang dilakukan, yaitu menentukan apakah suatu bangun segiempat adalah bujursangkar atau bukan. Sedangkan untuk lingkaran, ada operasi khusus untuk memperbesar/memperkecil lingkaran berdasarkan suatu faktor, misalnya x (ber-type *double*).

Aplikasi ini juga mempunyai struktur koleksi objek, yaitu *array* yang dapat digunakan untuk menampung beberapa objek jenis bangun tertentu, yaitu *array* of lingkaran, *array* of segiempat, dll. (**satu array hanya boleh menampung array dari satu jenis bangun saja**). *Array* yang digunakan merupakan *array* dinamis, yaitu ukuran *array* dapat berbeda-beda tergantung kebutuhan pengguna. Penggunaan elemen *array* selalu kontigu, dimulai dari indeks ke-0, dan disimpan indeks terakhir dari elemen yang efektif/terdefinisi. Operasi yang berlaku untuk semua jenis *array* sama (selain 4 sekawan), yaitu menampilkan semua objek dalam *array* ke layar.

a. Buatlah desain kelas untuk persoalan di atas dengan sedapat mungkin memanfaatkan konsep-konsep OO yang telah dijelaskan di kelas secara tepat. Gunakan diagram kelas untuk membuat desain aplikasi ini. Jika Anda memanfaatkan konsep-konsep spesifik OO seperti inheritance, kelas generik/template, ABC, polymorphism, dll., jelaskan alasannya.
b. Tuliskan file-file *.h yang merepresentasikan kelas-kelas yang Anda desain pada butir a secara lengkap dan beri komentar dengan baik. Tanpa komentar, fungsi-fungsi selain 4 sekawan tidak akan dinilai.

Soal 5. Implementator Kelas – Java (Bobot: 10%)

Buatlah implementasi kelas Point2D dalam bahasa Java yang merepresentasikan titik dalam koordinat 2 dimensi (x,y). Lengkapi implementasi kelas tersebut dengan kontruktor tanpa parameter (x,y = 0,0), kontruktor berparameter, getter, setter, dan prosedur cetak.

Buatlah implementasi kelas Point3D yang merepresentasikan titik dalam koordinat 3 dimensi (x,y,z) dan merupakan turunan dari Point2D.

IF2210 – UTS, Hari Rabu, Tanggal 16 Maret 2016 NIM/Nama/T.t.: 13514032 / Chalvin / chv

Prototype 4 sekawan untuk kelas X

```
X();                                  // ctor tanpa parameter
X(<list of parameter>);               // ctor dg parameter
X (const X&);                         // cctor
~X();                                 // dtor
X& operator= (const X& s);            // operator=
```

Soal1. Menuliskan kelas dengan inheritance yang baik dan benar

Diberikan header file sebagai berikut:

1. Lengkapilah header file dengan prototype sesuai dengan spesifikasi yang dituliskan pada komentar. Tambahkan keyword virtual, const sesuai dengan kaidah yang diajarkan untuk menjamin kode anda benar. Jika tidak perlu direalisasikan, tulis: //Tak Perlu
2. Tuliskan kode program Point.cpp dan Point3D.cpp sesuai dengan spesifikasi pada header file dan juga instansiasi semua variabel static. Di balik halaman 1 dan 2

```
//file Point.h
#ifndef _POINT_H
#define _POINT_H
class Point {
public:
// ctor tanpa parameter, yang akan menciptakan Point dg absis=0 dan ordinat=0
  Point();
// Tuliskan ctor yang akan menciptakan Point dengan absis = X dan Ordinat = Y yang diberikan
// sebagai parameter
  Point(int x, int y);
// cctor
  Point( const Point& P);
// dtor yang akan menjamin semua objek dibebaskan memorinya, dan berjalan dengan baik jika
// ada polymorphism
  Virtual ~Point();
// operator= untuk copy isi objek Point, bukan bitwise copy
Virtual Point& operator=(const Point& P);
//////////////////////// getter dan setter ////////////////////////
// Get absis, sebuah fungsi dengan nama GetAbsis dan mengirimkan sebuah integer
  Int GetAbsis();
// Get ordinat, sebuah fungsi dengan nama GetOrdinat dan mengirimkan sebuah integer
  int GetOrdinat();
// Set ordinat, dengan parameter sebuah integer
  void SetOrdinat(int Y);
// prosedur inverse
// membalik absis dan ordinat. Misalnya sebuah Point(4,5) akan menjadi (5,4)
  Virtual void inverse();
// fungsi yang akan mengembalikan nilai NbPoint
  Static int GetNbPoint();
private:
 int * Data; // Data[0] adalah absis, Data[1] adalah ordinat
 static int NbPoint; // banyaknya point yang pernah diciptakan
};
#endif
```

1

IF2210 – UTS, Hari Rabu, Tanggal 16 Maret 2016 NIM/Nama/T.t.: 135 14032 / Chalvin / Chal

```
//file Point3D.h
#ifndef _POINT3D _H
#define _POINT3D _H
class Point3D: public Point {
public:
// ctor tanpa parameter, yang akan menciptakan Point 3D dg absis=0 dan ordinat=0 dan color=0
    Point3D();
// Tuliskan ctor yang akan menciptakan Point dengan absis = X dan Ordinat = Y dan color=C
// X,Y dan C adalah parameter
    Point3D(int x, int y, int c);
// cctor
    Point3D(const Point3D& P);
// dtor yang akan menjamin semua objek dibebaskan memorinya, dan berjalan dengan baik jika
// ada polymorphism
    ~Point3D();
// operator= untuk copy isi objek Point3D, bukan bitwise copy
    Point3D& operator=(const Point3D P);
//////////////////////////// getter dan setter ////////////////////////
// Get color, sebuah fungsi dengan nama GetColor dan mengirimkan sebuah integer yaitu c
    int GetColor();
// Set color, dengan parameter sebuah integer yaitu color
    void SetColor(int c);
// prosedur inverse: absis dan ordinat dibalik, color dinegasi
// membalik absis dan ordinat. Misalnya sebuah Point(4,5, 1) akan menjadi (5,4,-1)
    void inverse();
// fungsi yang akan mengembalikan nilai NbPoint
    // tidak Perlu
private:
  int * c; // c[0] adalah "color", kode warna, 0 s.d. 256
};
#endif
```

1

0.25

IF2210 – UTS, Hari Rabu, Tanggal 16 Maret 2016 NIM/Nama/T.t.: 13514052 / Chalvin / du

Soal 2 Implementasi Operator overloading

Implementasikan operator+ dengan dua cara, yaitu dengan friend dan tanpa friend (sebagai fungsi anggota)

Operator+ akan menjumlahkan dua buah Vektor yang ukurannya sama, dengan hasil Vhasil[i]=V1[i]+V2[i]

```
//file Vektor.h
#ifndef _VEKTOR_H
#define _VEKTOR_H
class Vektor {
public:
// ctor , cctor, dtor dan operator= sudah dibuat dengan baik
// anda tidak perlu membuat prototype dan tak perlu merealisasi

// tulis prototype fungsi penjumlahan dua buah Vektor,
// menghasilkan Vektor sebagai friend function
friend Vector operator+ (const Vector& V1, const Vector& V2);
// tulis prototype prosedur penjumlahan dua buah Vektor,
// menghasilkan Vektor sebagai member function
Void operator+ (const Vector& V);
private:
  int * TabV;
  const int Max=100; // ukuran maksimum
};
#endif
```

IF2210 – UTS, Hari Rabu, Tanggal 16 Maret 2016 NIM/Nama/T.t.: 13514032 / Cholvin / cho

Soal 3 Reading Comprehension (Desain kelas)

a. Pada proses perancangan perangkat lunak berorientasi objek dikenal prinsip SOLID. Apa kepanjangan dari SOLID dan jelaskan secara singkat prinsip-prinsip tersebut.

b. Design pattern apakah yang menyediakan mekanisme untuk melakukan traversal terhadap elemen-elemen dari sebuah collection tanpa mengekspos representasi dari collection tersebut.

c. Design pattern apakah yang bisa mengendalikan banyaknya instance dari sebuah kelas dan menyediakan titik akses global terhadap instance tersebut.

d. Design pattern apakah yang memungkinkan satu atau lebih objek diberitahu (diberi notifikasi) bila terjadi perubahan state dari sebuah objek yang lain.

e. Design pattern apakah yang memungkinkan konversi/penggantian interface dari layanan/service yang diberikan ke sebuah client tanpa mengubah kelakuan (behavior) dari layanan tersebut.

UTS IF2210/Pemrograman Berorientasi Objek
Rabu, 6 Maret 2019
Hal. 1 dari 4

Waktu: 100 menit

NIM: 13517012 Kelas : 03
Nama: Johanes

Soal 1. Kelas dan Inheritance

Diberikan header file **X.h**, **Y.h**, dan **Z.h** sebagai berikut.

1. **Lengkapilah** semua header file di bawah ini dengan *prototype* (tanpa membuat *inline implementation*) sesuai dengan spesifikasi yang dituliskan pada komentar. Gunakan nama fungsi/prosedur sesuai yang diberikan di spesifikasi. Tambahkan keyword *virtual*, *const*, dan *static* sesuai dengan kaidah yang diajarkan untuk menjamin kode Anda benar. Jika anggota kelas tidak perlu direalisasikan, tuliskanlah: **//Tak Perlu** (Anda juga tidak perlu menuliskan *prototype*-nya).
2. Tuliskan kode program **X.cpp, Y.cpp**, dan **Z.cpp** sesuai dengan spesifikasi pada header file, termasuk inisialisasi semua variabel *static*, pada halaman kosong di balik **halaman 1 dan 2** (spesifikasi tidak perlu ditulis ulang).

```
// File: X.h
#ifndef _X_H_
#define _X_H_

class X {
    protected:
        int s;
        int a;
        int * tabdata;
        static int nbX; // banyaknya objek X yang sedang hidup pada suatu saat
    public:
        // default ctor: set nilai atribut sbb: s = 10; a = 0
        // alokasi tabdata dg ukuran s, dan inisialisasi semua elemen tabdata dg nilai = 0
        X();
        // user-defined ctor: set nilai atribut sbb: s = _s; a = _a
        // alokasi tabdata dg ukuran s, dan inisialisasi semua elemen tabdata dg nilai = _a
        X(int _s, int _a);
        // cctor: pastikan semua elemen tabdata tersalin dengan baik
        X(const X& x);
        // dtor: pastikan memori yang digunakan tabdata dibebaskan
        ~X();
        // operator assignment: pastikan semua elemen tabdata tersalin dengan baik
        X& operator=(const X& x);
        // getS: menghasilkan nilai atribut s
        int getS() const;
        // getA: menghasilkan nilai atribut a
        int getA() const;
        // getEl: menghasilkan nilai elemen atribut tabdata pada indeks ke-i
        int getEl(int i) const;
        // setS: mengubah nilai atribut s dg nilai baru
        void setS(int _s);
        // setA: mengubah nilai atribut a dg nilai baru
        void setA(int _a);
        // setEl: mengubah nilai elemen atribut tabdata pada indeks ke-i dg nilai baru
        void setEl(int i, int x);
        // print: mencetak nilai s, a, dan semua nilai elemen pada tabdata (format bebas)
        void print();
        // fx: menerima masukan sebuah integer, misalnya h
        // melakukan perubahan terhadap setiap elemen tabdata dg aturan yg tergantung pada
        // jenis objek X (belum bisa ditentukan di kelas ini)
        virtual void fx(int h) = 0;
        // getNbX: menghasilkan nilai nbX
        static int getNbX() const;
};
#endif
```

Waktu: 100 menit

NIM: 13519012 Kelas : 03
Nama: Johanes

```
// File: Y.h
#ifndef _Y_H_
#define _Y_H_

#include "X.h"

class Y : public X {
    private:
        int c;
    public:
        // default ctor: set nilai atribut a, s, dan tabdata spt pd default ctor X
        // dan set atribut c = 1
        Y();
        // user-defined ctor: set nilai atribut a, s, dan tabdata spt pd user-defined ctor X
        // dan set atribut c = _c
        Y(int _s, int _a, int _c);
        // cctor Y
        // Tdk Perlu
        // dtor Y
        // Tak Perlu
        // operator assignment Y
        // Tak Perlu.
        // getC: menghasilkan nilai atribut C
        int getC() const;
        // setC: mengubah nilai atribut c dg nilai baru
        void setC(int _c);
        // print: mencetak nilai s, a, dan semua nilai elemen pada tabdata dgn format sama spt pd X;
        // ditambah mencetak nilai c (format bebas)
        void Print();
        // fx: menerima masukan sebuah integer, misalnya h
        // melakukan perubahan terhadap setiap elemen tabdata dg aturan sbb:
        // misal nilai elemen ke-i adalah e[i]; maka nilai e[i] diubah mjd = e[i] * (c + h)
        void fx(int h);
};
#endif
```

```
// File: Z.h
#ifndef _Z_H_
#define _Z_H_

#include "X.h"

class Z : public X {
    public:
        // default ctor: set nilai atribut a, s, dan tabdata spt pd default ctor X
        // Tak perlu
        // user-defined ctor: set nilai atribut a, s, dan tabdata spt pd user-defined ctor X
        // Tak perlu
        // cctor Z
        // Tak Perlu
        // dtor Z
        // Tak perlu
        // operator assignment Z
        // Tak perlu
        // print: mencetak nilai s, a, dan semua nilai elemen pada tabdata dgn format sama spt pada X
        // Tak Perlu
        // fx: menerima masukan sebuah integer, misalnya h
        // melakukan perubahan terhadap setiap elemen tabdata dg aturan sbb:
        // misal nilai elemen ke-i adalah e[i]; maka nilai e[i] diubah mjd = e[i] * h
        void fx(int h);
};
#endif
```

Waktu: 100 menit

NIM: 13519012 Kelas : 03
Nama: Johanes

Soal 2. *Template* Kelas Dictionary

Diberikan *template* kelas entry sebagai berikut.

```
template<class K, class V>
class entry {
    private:
       K key;
       V value;
    public:
       entry(K k, V v): key(k), value(v) {}
       K getKey() { return key; }
       V getValue() { return value; }
       void setValue (V newValue) { value = newValue; }
};
```

Buatlah *template* kelas dictionary yang dapat berisi sekumpulan entry. Berikut contoh penggunaan *template* kelas dictionary. Implementasi *template* harus mencakup semua fungsi anggota (publik) yang pada contoh di bawah ditandai dengan cetak tebal dan garis bawah. Definisikan *exception* jika diminta.

```
int main() {
    dictionary<int,string> d; // dapat berisi sekumpulan entry<int,string>

    try {
        d.put(1,"one"); // isi d saat ini: [1:one]                        (menyimpan entry(1,"one") ke d)
        cout<<1<<": "<<d.get(1)<<endl; // mencetak "1: one"   (mengambil value entry yang memiliki key 1)

        d.put(2,"two"); // isi d saat ini: [1:one,2:two]
        cout<<1<<": "<<d.get(1)<<endl; // mencetak "1: one"
        cout<<2<<": "<<d.get(2)<<endl; // mencetak "2: two"

        d.put(1,"satu");  // isi d saat ini: [1:satu,2:two],     (menimpa value dari key 1 dengan "satu")
                          // BUKAN [1:one,2:two,1:satu]
        d.put(3,"three"); // isi d saat ini: [1:satu,2:two,3:three]
        cout<<1<<": "<<d.get(1)<<endl; // mencetak "1: satu"
        cout<<2<<": "<<d.get(2)<<endl; // mencetak "2: two"
        cout<<3<<": "<<d.get(3)<<endl; // mencetak "3: three"

        d.remove(2); // isi d saat ini: [1:satu,3:three]            (menghapus entry yang memiliki key 2)
        cout<<1<<": "<<d.get(1)<<endl; // mencetak "1: satu"
        if (d.containsKey(2)) {                // (mengembalikan true jika d memiliki entry dengan key 2)
            cout<<2<<": "<<d.get(2)<<endl; // tidak ada output
        }
        cout<<2<<": "<<d.get(2)<<endl; // mencetak "exception: key not found!"     (key 2 tidak ada di d)
        cout<<3<<": "<<d.get(3)<<endl; // tidak ada output karena sudah exception
    } catch (const char* c) {
        cout<<"exception: "<<c<<endl;
        return 1;
    }

    return 0;
}
```

Petunjuk:

- Anda tidak perlu membuat *method* 4 sekawan (dianggap sudah ada).
- Secara internal, Anda boleh menentukan sendiri bagaimana kumpulan entry disimpan (*array*, std::vector, dsb.). Petunjuk: jika menggunakan vector Anda tidak perlu memikirkan penggeseran elemen *array* bilamana terdapat elemen yang dihapus di tengah *array* (gunakan fungsi erase()).

Tuliskan jawaban di sisa halaman ini dan di halaman kosong di baliknya.

Waktu: 100 menit

NIM: 13517012 Kelas : 03
Nama: Johanes

Soal 3. Reading Comprehension

Diberikan program sebagai berikut, tuliskan hasil eksekusinya di balik halaman 4. Program dijamin lolos kompilasi.

```
#include <iostream>
using namespace std;

class A {
   public:
      A(const char i=0) {
         cout << "A::ctor" << endl;
         x = i;
      }
      A(const A& i) {
         cout << "A::cctor" << endl;
         x = i.x;
      }
      ~A() {
         cout << "A::dtor" << endl;
      }
      A& operator=(const A& i) {
         cout << "A::opr=" << endl;
         x = i.x;
         return *this;
      }
      void f() {
         cout << "A::f()" << endl;
      }

   private:
      char x;
};

class B {
   public:
      B(const int n) {
         cout << "B::ctor" << endl;
         pa = new A[n];
         this->n = n;
      }
      B(const B& a) {
         cout << "B::cctor" << endl;
         n = a.n;
         pa = new A[n];
      }
      B& operator=(const B& b) {
         cout << "B::opr=" << endl;
         if (this != &b) {
            delete[] pa;
            n = b.n;
            pa = new A[n];
         }
         return *this;
      }
      virtual ~B() {
         cout << "B::dtor" << endl;
         delete[] pa;
      }
      virtual void f() {
         cout << "B::f()" << endl;
         for (int i=0; i<n; i++)
            pa[i].f();
      }

   private:
      A *pa;
      int n;
};
```

```
class C: public A, public B {
   public:
      C(const char n): B(n) {
         cout << "C::ctor" << endl;
      }

      virtual ~C() {
         cout << "C::dtor" << endl;
      }

      virtual void f() {
         cout << "C::f()" << endl;
         A::f();
         B::f();
      }
};

int main() {

   cout << "1:" << endl;
   A *pa;
   B *pb;
   C *pc = new C(3);

   cout << "2:" << endl;
   pa = pc;
   pb = pc;
   pa->f();
   pb->f();

   cout << "3:" << endl;
   pc->B::f();

   cout << "4:" << endl;
   delete pb;

   cout << "5:" << endl;
   C c1(2);
   C c2 = c1;

   cout << "6:" << endl;
   c2 = c1;

   cout << "0:" << endl;

   return 0;
}
```

UTS SEMESTER II – 2020/2021

Section 1

Soal di Quizizz yang tidak sempat didokumentasikan

Section 2

Soal 1. Inheritance, Abstract Class/Function

A. Lengkapi header kelas abstrak Property berikut ini dengan *prototype* (tanpa membuat *inline implementation*) sesuai dengan spesifikasi yang dituliskan pada komentar. Gunakan nama fungsi/prosedur sesuai yang diberikan di spesifikasi. Tambahkan *keyword* virtual, const, dan/atau static sesuai dengan kaidah yang diajarkan untuk menjamin kode Anda benar. Jika anggota kelas tidak perlu direalisasikan, tuliskanlah: // TIDAK PERLU (Anda juga tidak perlu menuliskan *prototype*-nya).

```
// Class Property
// File: Property.h

#ifndef PROPERTY_H
#define PROPERTY_H

#include <string>
using namespace std;

class Property {
  protected:
  // Atribut
    string name;
    string type; //hotel; hostels; villas; cottages
    int openYear;
  public:
    // User-defined constructor: set nilai atribut berdasarkan nilai
    // parameter masukan

    // Default constructor: set nilai atribut sbb:
    // name = "noname"; openYear = 1900; type = "none"

    // Copy constructor

    // Destructor

    // Operator Assignment

    // ... set_name(...)

    // ... get_name()

    // ... set_type(...)

    // ... get_type()

    // ... get_age(), asumsikan tahun saat ini dapat diakses dengan
```

```
    // makro CURRENT_YEAR

    // ... displayInfo(): Mencetak nama, umur, type, dan rate Property

    // ... rate(): menghitung biaya property sesuai dengan umur dan
    // tergantung type property

};

#endif // PROPERTY_H
```

B. Buatlah *subclass* dengan mewariskan kelas `Property` pada kelas `Hotel` dan `Hostel`, dengan ketentuan sebagai berikut (buat *overload function* jika diperlukan):
   a. Kelas `Hotel` mencatat jumlah star setiap Hotel, diakses menggunakan `set_star()` dan `get_star()`. Override `displayInfo()` untuk mencetak informasi tambahan yaitu *total rate* yang didapatkan dari perkalian antara rate dengan star.
   b. Kelas `Hostel` mencatat jumlah facility yang sudah digunakan (`set_facility()`, `get_facility()`) dan memiliki fungsi `calculateFacility()` yang menghasilkan *expense* yaitu jumlah facility dikalikan dengan 80000. *Override* `displayInfo()` untuk mencetak informasi tambahan yaitu *expense*.

C. Tuliskan implementasi kelas abstrak `Property` dalam `Property.cpp` sesuai dengan spesifikasi pada *header file*.

D. Lengkapi `main.cpp` berikut pada bagian yang kosong untuk menguji perilaku kelas-kelas yang Anda buat dengan:

```
#include "Property.h"
#include <iostream>
using namespace std;

int main() {
  ____________ hilton = Hotel("Hotel Hilton", 2000);
  ____________ vio = Hostel("Hostel Vio", 2003);
  // cetak semua property dan informasi semua property
  // [gunakan Property::displayInfo()]
    ____________________________________
    ____________________________________

}
```

Soal 2. Queue dan PriorityQueue

Queue adalah sebuah struktur data yang dapat menyimpan sekumpulan elemen, dimana sebuah elemen dimasukkan dan dikeluarkan secara *first-in-first-out*. Sebuah queue umumnya memiliki dua method penting, yaitu *enqueue* untuk memasukkan elemen ke posisi paling akhir, dan *dequeue* untuk mengeluarkan elemen pada posisi paling depan. Berikut ini adalah contoh serangkaian aksi *enqueue* dan *dequeue* pada sebuah queue dengan elemen bertipe integer dan memiliki kapasitas 10.

No.	Aksi	Kondisi Queue
0	Kondisi awal kosong	Elemen:

1	enqueue(8)	Elemen: 8
2	enqueue(6)	Elemen: 8, 6
3	enqueue(4)	Elemen: 8, 6, 4
4	dequeue(), returns 8	Elemen: 6, 4
5	enqueue(2)	Elemen: 6, 4, 2
6	dequeue(), returns 6	Elemen: 4, 2

Struktur queue ini dapat di-*extend* lebih lanjut menjadi apa yang disebut PriorityQueue. Pada PriorityQueue, setiap elemen memiliki prioritas yang menentukan posisi elemen tersebut pada queue. Semakin tinggi prioritas sebuah elemen, akan semakin di depan posisinya. Elemen-elemen dengan prioritas yang sama, akan diperlakukan secara *first-in-first-out* sebagaimana pada Queue biasa. Berikut ini adalah contoh serangkaian aksi *enqueue* dan *dequeue* pada sebuah PriorityQueue dengan elemen bertipe integer dan memiliki kapasitas 10. Fungsi *enqueue* memiliki dua parameter, yaitu enqueue(elemen, prioritas).

No.	Aksi	Kondisi Queue
0	Kondisi awal kosong	Elemen: Prioritas:
1	enqueue(8, 1)	Elemen: 8 Prioritas: 1
2	enqueue(6, 2)	Elemen: 6, 8 Prioritas: 2, 1
3	enqueue(4, 1)	Elemen: 6, 8, 4 Prioritas: 2, 1, 1
4	dequeue(), returns 6	Elemen: 8, 4 Prioritas: 1, 1
5	enqueue(2, 3)	Elemen: 2, 8, 4 Prioritas: 3, 1, 1
6	dequeue(), returns 2	Elemen: 8, 4 Prioritas: 1, 1

Sebuah struktur data Queue dan PriorityQueue dapat dibuat generik sehingga elemennya dapat bertipe apapun. Dengan bahasa C++,

A. Buatlah sebuah kelas Queue generik dengan kapasitas maksimal 10 elemen.
B. Turunkan kelas Queue tersebut menjadi sebuah kelas `PriorityQueue` generik.

Kelas-kelas tersebut minimal harus memiliki:

a. Default constructor
b. Destruktor
c. Method enqueue, untuk memasukkan sebuah elemen ke dalam queue. Method ini akan melempar sebuah exception jika queue sudah penuh.
d. Method dequeue, untuk mengeluarkan sebuah elemen ke dalam queue. Method ini akan melempar sebuah exception jika queue kosong.
e. Method `print`, untuk menampilkan isi queue (bentuk tampilan bebas)

C. Tulislah juga sebuah *main function* yang membuat sebuah `Queue` dan `PriorityQueue` dengan isi seperti contoh di atas. Gunakanlah *exception handler* di *main function* untuk menangani *exception* yang dilemparkan oleh kelas `Queue` dan `PriorityQueue`.

Anda boleh menggunakan kerangka program berikut ini. Jika ada bagian kerangka program yang kurang lengkap atau salah, Anda dapat melengkapinya atau menggantinya.

```
#include <iostream>

using namespace std;

template <class T>
class Queue {
  private:
    T *elements;
    int nElements;
  public:
    Queue();
    ~Queue();
    void enqueue(const T&);
    T dequeue();
    void print();
};

// Default constructor

// Destructor

// Method enqueue

// Method dequeue

// Method print


template <class T>
class PriorityQueue : public Queue<T> {
  private:
    int *priorities;
  public:
    PriorityQueue();
    ~PriorityQueue();
    void enqueue(const T&, int);
};
```

```
// Default constructor

// Destructor

// Method enqueue

// Method dequeue, override if needed

// Method print, override if needed


// Main function
int main() {

}
```

Soal 3. STL

Pada Python, kita dapat membuat *tuple*, yakni gabungan dari beberapa nilai, misalnya (123, "abc", true). Di C++, kita juga dapat menggunakan STL `pair<>` untuk menggabungkan dua nilai. Namun, kali ini Anda membutuhkan tuple untuk tiga nilai. Karena itu, buatlah sebuah kelas bernama `Triplet` yang dapat menampung tiga nilai dengan tipe data yang berbeda.

Contoh Kode	Contoh Output

`Triplet<int, int, int> a(3, 2, -5);`
 	
`cout << a.getFirst() << endl;`
`cout << a.getSecond() << endl;`
`cout << a.getThird() << endl;`
	`3`
	`2`
	`-5`

`Triplet<string, float, string> a("abc", 4.5, "def");`
`Triplet<string, float, string> b("ghi", -1.0, "def");`
`Triplet<string, float, string> c("abc", 4.5, "def");`
 	
`if (a == b) {`
`  cout << "a == b" << endl;`
`}`
`if (a == c) {`
`  cout << "a == c" << endl;`
`}`
	`a == c`

SECTION I: SOAL KONSEP

1. Bila kita memerlukan fungsi yang melakukan operasi sejenis terhadap tipe data yang berbeda, maka kita menggunakan:
    a. virtual function
    b. template (/generic) function
    c. friend function
    d. const function
    e. cctor
2. (Multiple Answer)Berikut ini yang merupakan jenis-jenis inheritance pada C++ adalah
    a. Hierarchical inheritance
    b. Double inheritance
    c. Hybrid Virtual inheritance
    d. Multilevel inheritance
3. (Multiple Answer)Jika kelas D1 dan D2 merupakan turunan dari kelas B yang mengandung definisi method virtual M, dan kelas E yang diturunkan dari D1 dan D2 tidak meng-override M, maka:
    a. Salah satu dari D1 atau D2 harus meng-override M
    b. D1 dan D2 keduanya harus meng-override M
    c. Salah satu dari D1 atau D2 boleh meng-override M
    d. D1 dan D2 tidak perlu meng-override M
4. (Multiple Answer)Mana sajakah pernyataan yang benar terkait exception handling di C++
    a. Bekerja dengan mengubah alur eksekusi program sambil mengembalikan objek tertentu sebagai informasi alur yang baru
    b. Jika sebuah method ditulis menangani exception, invokasi sebaiknya dilakukan dalam sebuah blok try ... finally
    c. Error selalu harus ditangani dengan exception handling
    d. Exception dapat menyebabkan program terminate abnormally
5. (Multiple Answer)Diberikan screenshot program terlampir. Pilih pernyataan yang benar:

```
1 #include <iostream>
2 using namespace std;
3
4 class Kendaraan {
5 public:
6     virtual void info(){
7         cout << "Ini adalah kendaraan."<< endl;
8     }
9 };
10
11 class Mobil:virtual public Kendaraan {
12 public:
13     void info(){
14         cout << "Ini adalah kendaraan mobil."<< endl;
15     }
16 };
17
18 class Motor:virtual public Kendaraan {
19  public:
20     void info(){
21         cout << "Ini adalah kendaraan motor."<< endl;
22     }
23 };
24
25 class Batmobile:public Mobil, public Motor {
26 public:
27     void info(){
28         cout << "Ini adalah kendaraan mobil motor Batmobile."<< endl;
29     }
30 };
31
32 int main(){
33     Kendaraan kendaraan;
34     Mobil mobil;
35     Motor motor;
36     Batmobile batmobile;
37
38     Kendaraan* obj1 = &kendaraan;
39     obj1->info();
40     obj1 = &mobil;
41     obj1->info();
42     obj1 = &motor;
43     obj1->info();
44     obj1 = &batmobile;
45     obj1->info();
46
47     Kendaraan obj2 = kendaraan;
48     obj2.info();
49     obj2 = mobil;
50     obj2.info();
51     obj2 = motor;
52     obj2.info();
53     obj2 = batmobile;
54     obj2.info();
55
56     return 0;
57 }
```

a. Baris ke-41 dan ke-43 akan menampilkan string "Ini adalah kendaraan mobil." dan "Ini adalah kendaraan motor.".
b. Baris ke-45 menyebabkan program error.
c. Baris ke-45 akan menampilkan string "Ini adalah kendaraan.".
d. Baris ke-54 menyebabkan program error.
e. Baris ke-54 akan menampilkan string "Ini adalah kendaraan.".

6. (Multiple Answer)Berikut ini merupakan prinsip dari OOP
   a. Monomorfisme
   b. Encapsulation
   c. Abstraction
   d. Inheritance

7. Container dengan struktur sekuens berindeks yang memperbolehkan menambah dan menghapus elemen di awal dan di akhir disebut sebagai
    a. iterator
    b. deque
    c. queue
    d. bidirectional
8. (Multiple Answer)Yang harus ada pada sistem pemrograman berorientasi objek
    a. Objek
    b. Kelas
    c. Method
    d. Inheritance
9. (Multiple Answer)Berikut adalah karakteristik kelas abstrak (abstract base class/ABC)
    a. Kita tidak dapat membuat objek dari kelas ABC
    b. Kelas ABC dapat langsung diinstansiasi
    c. Objek yang diinstansiasi terbatas pada instansiasi kelas turunan ABC
    d. Kelas ABC setidaknya memiliki satu method yang tidak diimplementasikan
    e. Suatu kelas dikatakan kelas ABC jika terdapat method yang dioverride oleh kelas turunannya
10. (Multiple Answer)Manakah penjelasan yang tepat untuk jenis iterator berikut
    a. Output: iterator yang menulis elemen dari container dan bisa bergerak maju atau mundur
    b. Forward: iterator untuk membaca/menulis elemen dari container, hanya bisa bergerak maju, tapi bisa multipass
    c. Random access: iterator yang dapat maju, tetapi bisa juga mundur, dan juga lompat ke elemen manapun
    d. Input: iterator yang membaca elemen dari container dan hanya bisa bergerak maju
11. (Multiple Answer)Pilihlah pernyataan yang benar mengenai beberapa jenis objek dalam program C++
    a. Free Store Object
    b. Member Object
    c. Dynamic Object
    d. Dependent Object
12. (Multiple Answer)Jika kita membutuhkan kelas Stack yang ketika setiap diinstansiasi dapat menampung tipe data yang berbeda, maka kita bisa memanfaatkan:
    a. generic class
    b. standard template library
    c. polymorphism dari base class Array
    d. struct
    e. operator overloading

SECTION II: SOAL PEMROGRAMAN

Soal 1. Inheritance, Abstract Class/Function

a. Lengkapi header kelas abstrak `Hotel` berikut ini dengan prototype (tanpa membuat *inline implementation*) sesuai dengan spesifikasi yang dituliskan pada komentar. Gunakan nama fungsi/prosedur sesuai yang diberikan di spesifikasi. Tambahkan keyword `virtual`, `const`, dan/atau `static` sesuai dengan kaidah yang diajarkan untuk menjamin kode Anda benar. Jika anggota kelas tidak perlu direalisasikan, tuliskanlah:

`// TIDAK PERLU` (Anda juga tidak perlu menuliskan *prototype*-nya).

```
// Class Hotel
// File: Hotel.h

#ifndef HOTEL_H
#define HOTEL_H

#include <string>
using namespace std;

class Hotel {
  protected:
  // Atribut
    string name;
    string bintang; //Melati; bintang_tiga; bintang_empat; bintang_lima
    int openYear;
  public:
    // User-defined constructor: set nilai atribut berdasarkan nilai parameter masukan

    // Default constructor: set nilai atribut sbb:
    // name = "noname"; openYear = 1900; bintang = "none"

    // Copy constructor

    // Destructor

    // Operator Assignment

    // ... set_name(...)

    // ... get_name()

    // ... set_bintang(...)

    // ... get_bintang()

    // ... get_age(), asumsikan tahun saat ini dapat diakses dengan makro CURRENT_YEAR

    // ... displayInfo(): Mencetak nama, umur hotel, bintang, dan room_rate

    // ... rate(): menghitung biaya menginap sesuai dengan umur dan tergantung bintang hotel


};

#endif // HOTEL_H
```

b. Buatlah *subclass* dengan mewariskan kelas `Hotel` pada kelas `bintang_empat` dan `bintang_lima`, dengan ketentuan sebagai berikut (buat *overload function* jika diperlukan):
   - Kelas `bintang_empat` mencatat jumlah star layanan setiap hotel bintang empat, diakses menggunakan `set_star()` dan `get_star()`. *Override* `displayInfo()` untuk mencetak informasi tambahan yaitu totalRate yang didapatkan dari perkalian antara rate dengan star.
   - Kelas `bintang_lima` mencatat jumlah facility yang sudah digunakan (`set_facility()`, `get_facility()`) dan memiliki fungsi `calculateFacility()` yang menghasilkan expense yaitu jumlah facility dikalikan dengan 100000. *Override* `displayInfo()` untuk mencetak informasi tambahan yaitu expense.

c. Tuliskan implementasi kelas abstrak `Hotel` dalam `Hotel.cpp` sesuai dengan spesifikasi pada header file.

d. Lengkapi `main.cpp` berikut pada bagian yang kosong untuk menguji perilaku kelas-kelas yang Anda buat dengan:

```
#include "Hotel.h"
#include <iostream>
using namespace std;

int main() {

    ____________ aston = bintang_empat("Hotel Aston", 2010);

    ____________ padma = bintang_lima("Hotel Padma", 2000);

    // cetak semua hotel dan informasi semua hotel
    // [gunakan Hotel::displayInfo()]

        ____________________________________

        ____________________________________

}
```

Soal 2. Generic Class dan Exception Handling

Berikut ini adalah contoh serangkaian aksi enqueue dan dequeue pada sebuah queue dengan elemen bertipe integer dan memiliki kapasitas 3 elemen.

No.	Aksi	Kondisi Queue
0	Kondisi awal kosong	`Elemen:`
1	dequeue()	`Elemen:` `Queue telah kosong`
2	enqueue(8)	`Elemen: 8`
3	enqueue(6)	`Elemen: 8, 6`
4	enqueue(4)	`Elemen: 8, 6, 4`
5	enqueue(9)	`Elemen: 8, 6, 4` `Queue telah penuh`
6	dequeue(), returns 8	`Elemen: 6, 4`
7	enqueue(2)	`Elemen: 6, 4, 2`
8	dequeue(), returns 6	`Elemen: 4, 2`

Sebuah struktur data Queue dapat dibuat generik sehingga elemennya dapat bertipe apapun.

Tugas kalian adalah:

1. Dengan bahasa C++, buatlah sebuah kelas Queue generik dengan kapasitas maksimal 3 elemen. Kelas tersebut minimal harus memiliki:

    a. Default constructor.
    b. Destructor.
    c. Method enqueue, untuk memasukkan sebuah elemen ke dalam Queue. Method ini akan melempar sebuah exception jika Queue sudah penuh.
    d. Method dequeue, untuk mengeluarkan sebuah elemen ke dalam Queue. Method ini akan melempar sebuah exception jika Queue kosong.
    e. Operator overloading << untuk menampilkan isi Queue (bentuk tampilan bebas).

2. Membuat kelas abstrak QueueException dengan method what() yang mengembalikan informasi tipe exception. Kelas turunan QueueException dapat menangani 2 jenis exceptions:

    a. Suatu Queue telah kosong.
    b. Suatu Queue telah penuh.

3. Tulislah main program yang membuat sebuah Queue dengan isi seperti contoh di atas. Gunakanlah exception handler untuk menangani exception yang dilemparkan oleh kelas Queue.
4. Tunjukkan bahwa struktur data Queue telah dibuat generik dengan mampu menampung elemen yang beragam:

    a. Minimal 2 tipe data “built-in”, yakni Queue of int dan Queue of float.
    b. Minimal 1 tipe data “customized”, yakni Queue of Mobil (definisi kelas Mobil dibebaskan).

Gunakan kerangka program berikut ini. Program dikerjakan dalam satu buah file main.cpp untuk sekedar alasan kepraktisan dalam pemeriksaan ujian. Jika ada bagian kerangka program yang kurang lengkap atau salah, Anda dapat melengkapinya atau menggantinya.

```
#include <iostream>

using namespace std;

template <class T>
class Queue {
  private:
    // lengkapi private member di sini
    T *elements;
    ...
  public:
  // lengkapi public member di sini
   ...

};

// Default constructor
// Destructor
// Method enqueue
// Method dequeue
// Operator overloading <<
```

```
// Main function
int main() {



}
```

Soal 3. STL

Di negara NeverHeardBefore, pemimpin pemerintahan memiliki masa jabatan yang ditentukan oleh voting. Seorang pemimpin memiliki nilai diri tertinggi dibandingkan dengan birokrat lainnya. Setiap pemimpin/ birokrat memiliki dua birokrat lainnya untuk membantu, dengan ketentuan nilai diri mereka lebih tinggi dari dua asistennya tersebut. Struktur seperti ini dikenal dengan max heap yang merupakan implementasi dari priority queue. Jika pemimpin sekarang di-*vote* untuk berhenti memimpin, maka penggantinya diambil dari salah satu asisten yang memiliki skor tertinggi. Gunakan STL untuk mengimplementasikan hal berikut:

1. Membuat sebuah heap yang terdiri dari 50 bilangan integer random untuk merepresentasikan struktur pemimpin/ birokrat negara NeverHeardBefore
2. Menambahkan birokrat baru dengan nilai diri integer random
3. Melakukan vote terhadap pemimpin (menghapus elemen maksimum dari heap)
4. Mengurutkan struktur birokrat/ heap tersebut.

Note: kalau butuh contoh heap http://www.cplusplus.com/reference/queue/priority_queue/

atau dapat juga memanfaatkan vector

UAS SEMESTER II – 2012/2013

IF2032/Pemrograman Berorietasi Objek

Tanggal: 7 Mei 2013, 120 menit, closed book

Bagian I. C++ (Bobot total: 30 %)

Soal 1. Polyline (Bobot: 20%)

Suatu polyline adalah suatu garis dengan segmen yang dibentuk dari titik-titik (Point). Didefinisikan file Point.h dan Polyline.h serta implementasi Polyline dalam Polyline.cpp sebagai berikut:

```
// File:Point.h
#ifndef _POINT_
#define _POINT_
Class Point {
  private:
    int x; // absis
    int y; // ordinat
  public:
    Point();
    Point(int _x, int _y);
    void Cetak(); // mencetak Point ke layar dalam bentuk (x,y)
};
#endif
```

```
// File:Polyline.h
#ifndef _POLYLINE_
#define _POLYLINE_
Class Polyline {
  private:
    Point *points;      // array of Point membentuk Polyline
    int max;            // maksimum banyaknya Point yang bias ditampung dalam
                        // sebuah Polyline
    int nbpoints;       // banyaknya Point yang ditampung Polyline
                        // array of Point diisi dari indeks ke-0 s.d.
                        // nbpoints-1
    ..............      // (1.1) Deklarai nbpoly
  public:
    Polyline();
    Polyline(int _max); // menciptakan Polyline dengan max = _max
    ~Polyline();
    Polyline (const Polyline& PL);
    Polyline& operator= (const Polyline& PL);
    void Cetak();       // mencetak seluruh Point dalam Polyline ke layar
    ..............      // (1.2) Deklarasi operator overloading +
    ..............      // (1.3) Deklarasi getNbPoly
};
#endif
```

Catatan: Point.cpp diasumsikan sudah diimplementasikan. Untuk polyline.cpp dibawah ini, fungsi-fungsi yang dituliskan, diasumsikan sudah diimplementasikan dan tidak perlu dikerjakan.

```
//File:Polyline.cpp
#include <iostream>
using namespace std;
#include "Polyline.h"

.............. // (1.4) Inisiasi nbpoly
```

```
Polyline::Polyline() { … }

.............. // (1.5) Implementasi konstruktor user-defined Polyline
.............. // (1.6) Implementasi destructor Polyline
.............. // (1.7) Implementasi copy constructor Polyline

Polyline::Polyline& operator= (const Polyline& PL) { … }
void Polyline::Cetak() { … }

.............. // (1.8) Implementasi operator overloading +
.............. // (1.9) Implementasi getNbPoly
```

Isilah titik-titik di atas (dengan cara menuliskan jawaban sesuai nomor) sesuai dengan petunjuk berikut ini:

1.1. Deklarasikan **nbpoly** yang merupakan suatu nilai global yang digunakan untuk menampung berapa banyak objek Polyline yang sedang *hidup* (pernah diciptakan dan belum dimusnahkan) pada saat *runtime*.
1.2. Deklarasikan **operator overloading +** yang digunakan untuk menambahkan sebuah Point, misalnya P, ke dalam suatu Polyline, misalnya **L**, sehingga dapat di-invoke dalam bentuk L + P dan P + L. Berikan komentar dengan baik.
1.3. Deklarasikan fungsi **getNbPoly** yang mengeluarkan nilai nbpoly.
1.4. Tuliskan kode untuk **menginisialisasi nbpoly** dengan 0.

Untuk implementasi kode di bawah ini (nomor 1.5 s.d. 1.9) tuliskan seluruh kode termasuk signature fungsinya. Ingat juga untuk menuliskan kode untuk meng-*update* nilai nbpoly jika diperlukan.

1.5. Tuliskan kode **konstruktor user-defined Polyline** dengan parameter masukan _max untuk menciptakan Polyline dengan ukuran maksimum _max. Gunakan sedapat mungkin *constructor initialization list*.
1.6. Tuliskan kode **destruktor Polyline**, dengan mengingat bahwa array yang sudah diciptakan harus dihancurkan.
1.7. Tuliskan kode **copy constructor Polyline.**
1.8. Tuliskan kode **operator overloading +** sesuai deskripsi pada nomor 1.2.
1.9. Tuliskan kode **getNbPoly** sesuai deskripsi pada nomor 1.3.

Soal 2. Animal (Bobot: 10%)

Tuliskan hasil eksekusi bagian dari kode di bawah ini dengan menuliskan pada nomor yang sesuai. Jika bagian kode tidak mungkin dieksekusi, tuliskan “Error” dan jelaskan mengapa.

```
//File:Animal.cpp
#include <iostream>
using namespace std;

class Animal {
  public:
    virtual void greeting() = 0;
};
class Cat: public Animal {
  public:
    void greeting()            { cout << “Meow!” << endl; }
};
class Dog: public Animal {
  public:
    virtual void greeting()    { cout << “Woof!” << endl; }
```

```
    void greeting(Dog another) { cout << "Wooooooooof!" << endl;
};
class BigDog: public Dog {
  public:
    void greeting()            { cout << "Woww!" << endl; }
    void greeting(Dog another) { cout << "Wooooowwwww!" << endl;
};
int main() {
  Animal animal; animal1.greeting();      //(2.1) ....................
  Cat cat1; cat1.greeting();              //(2.2) ....................
  Dog dog1; dog1.greeting();              //(2.3) ....................
  Bigdog bigdog1; bigdog1.greeting();     //(2.4) ....................
  dog1 = bigdog; dog1.greeting();         //(2.5) ....................
  BigDog bigdog2; dog1.greeting(bigdog2); //(2.6) ....................
  bigdog2.greeting(bigdog1);              //(2.7) ....................
  bigdog2 = dog1; bigdog2.greeting;       //(2.8) ....................
  return 0;
}
```

Bagian 2. JAVA (Bobot total: 70%)

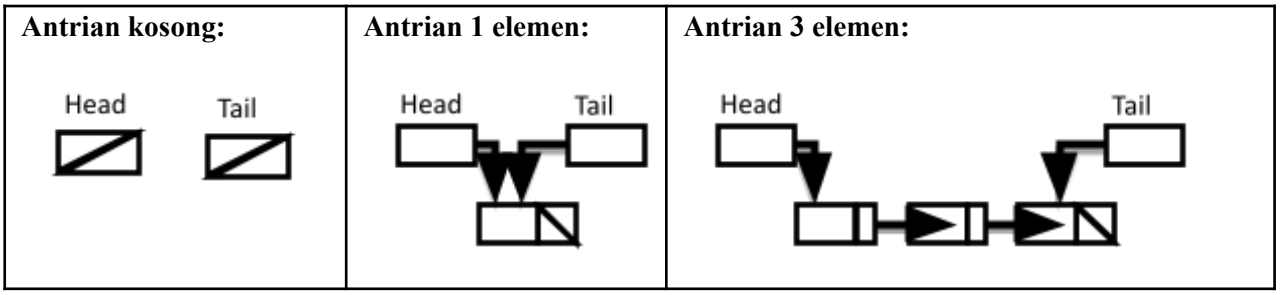
Soal 1. Konsep Umum (Bobot: 20%)

a. Jelaskan apa yang dimaksud **interface** dalam bahasa Java, dan buatlah contoh penggunaanya.

b. Jelaskan apa yang dimaksud dengan **polymorphism** dan buatlah contohnya.

Soal 2. Queue Generik (Bobot: 25%)

Sebuah kelas generik **GQueue** merepresentasikan antrian, dan dikenali alamat pertamanya bernama **Head** dan alamat terakhirnya bernama **Tail.** Antrian tidak kosong mengandung elemen-elemen bertipe generik **GElement.** Sebuah elemen antrian mengandung variabel **info** yang bertipe generik, dan sebuah anddress **ext** yang menunjuk kepada elemen berikutnya (jika ada). Perhatikan bahwa antrian ini diimplementasi dalam bentuk list linier, dan bukan tabel kontigu. Berikut adalah ilustrasi antrian kosong, antrian dengan 1 elemen dan antrian dengan 3 elemen.



Skema umum kelas GElement adalah sbb:

```
public class GElement ...... {
  ......... info;
  ......... next;

  // konstruktor, dengan menset nilai info dengan newInfo, dan next
  public Gelement (...... newInfo);
  public ......... getNext();
  public ......... getInfo();
  public ......... setInfo(...... newInfo);
  public ......... setNext(...... newAddress);

  // cetak info ke layar
  public void print();
```

```
}
```

Kelas GQueue mengandung method dengan spesifikasi sbb:

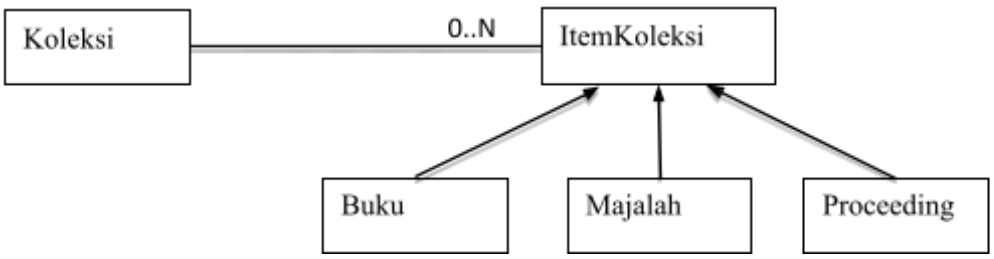
```
// Deklarasi nama kelas

// Deklarasi ember variabel Head

// Deklarasi ember variabel Tail

// konstruktor

// method IsEmpty
// menghasilkan tua jika antrian kosong

// method Insert(parameter e adalah elemen GElement)
// IS: lis mungkin kosong
// FS: e ditambahkan sebagai elemen terakhir queue, Tail diupdate

// fungsi Delete:
// mengembalikan elemen generik GElement yang merupakan elemen pertama
// antrian
// prakondisi: anrian tidak kosong
// fungsi menghapus elemen pertama antrian, mengupdate had, dan
// mengembalikan // elemen pertama tersebut sebagai return value
```

Implementasi kelas GElement dan GQueue di atas, dan program utama MainQueue dalam bahasa Java dengan skema sbb:

```
public static void main(String[] args) {
  // Deklarasi dan inisialisasi GQueue QInt dengan elemen Integer

  // Deklarasi dan inisialisasi GElement e1 dengan tipe spesifik Integer,
  // dengan nilai 10
  // Deklarasi dan inisialisasi GElement e2 dengan tipe spesifik Integer,
  // dengan nilai 20

  // insert e1 ke QInt
  // insert e2 ke QInt

  // Deklarasi GElement e3 dengan tipe spesifik Integer

  // Assign e3 dengan hasil invoke Delete dari QInt

}
```

Soal 3. Implementator kelas (Bobot: 25%)
Perhatikan bahwa, secara prinsip persoalan berikut mirip dengan salah satu Soal UTS IF2032 yang lalu.

Sebuah sistem **perpustakaan** berbayar memiliki **koleksi** berbagai **item koleksi** perpustakaan, yang terdiri atas **buku, majalah,** dan **proceeding** (kumpulan paper konferensi/seminar). Setiap item koleksi memiliki identitas/id yang unik dan untuk satu judul diasumsikan hanya ada 1 eksemplar item. Untuk memodelkan koleksi yang dimiliki perpustakaan, telah didesain kelas diagram di bawah ini. Perhatikan bahwa dalam persoalan ini, anggota yang dimiliki perpustakaan tidak dimodelkan dan tidak menjadi bagian dari sistem ini.



Perhatikan diagram di atas. Koleksi terdiri atas nol atau lebih ItemKoleksi. ItemKoleksi adalah sebuah kelas abstrak, dan diturunkan menjadi kelas konkret Buku, Majalah, dan Proceeding. Jumlah koleksi yang dapat dikelola, maksimum 1000 item koleksi. ItemKoleksi memiliki atribut id (integer), judul (string), dan tahun terbit (integer), sedangkan Majalah memiliki atribut tambahan berupa bulanTerbit (integer).

ItemKoleksi memiliki fungsi dan prosedur sbb.:

1. **Fungsi abstrak HitungBiayaPeminjaman (input lamaPinjam: integer)** yang menghitung dan mengembalikan biaya peminjaman dengan kebijakan yang berbeda tergantung jenis itemnya sbb:

No	Jenis ItemKoleksi	Biaya Peminjaman
1	Buku	LamaPeminjaman*Rp. 500
2	Majalah	LamaPeminjaman*RP. 1500*(1+(1/TahunTerbit))
3	Proceeding	Rp. 5000 (tidak tergantung dengan lama peminjaman)

2. **Prosedur PrintInfo()**, yang mencetak nilai semua atribut yang dimiliki oleh item koleksi.

Koleksi memiliki prosedur sbb.:

1. **procedure AddItem (input item: ItemKoleksi)**, yang menambahkan item baru pada table item yang dikelola oleh koleksi dan merupakan **array static**.
2. **Procedure Print()**, yang mencetak nilai seluruh atribut yang dimiliki oleh semua item yang dikelola oleh koleksi tersebut.

Tugas Anda adalah:

1. Tuliskan **Koleksi.java, ItemKoleksi.java, dan Majalah.java,** yang merepresentasikan kelas-kelas sesuai penjelasan di atas.
2. Anggaplah semua kelas yang didesain sudah diimplementasikan secara lengkap, buatlah kelas **mainLib.java** yang memiliki perilaku sebagai berikut:
   a. Menambahkan Item koleksi berupa Buku dengan ItemId = 1, Judul = “Object Oriented Programming”, tahun terbit = 2010.
   b. Menambahkan Item koleksi berupa Majalah dengan ItemId = 2, Judul = “Informatics Meagazine”, tahun = 2012, bulan terbit = 2.
   c. Mencetak semua item (buku, majalah, proceeding yang dikelola koleksi).